PRISM: A CPU/GPU Portfolio Optimization Engine for Deadline-Bounded Institutional Rebalancing

Debdoot Ghosh

Asymmetry Computing

Email debdoot.ghosh23@alumni.imperial.ac.uk

Web asymmetrycomputing.com

June 2026

Abstract

Institutional rebalancing is a batched optimization workload with a hard operating deadline: hundreds of accounts need new weights under budget, turnover, exposure, exclusion, and tax-aware controls before trading can proceed. This paper evaluates PRISM, a proprietary CPU/GPU portfolio optimization engine, through a public evaluation boundary; problem data in, and returned weights, status codes, timings, memory class, external feasibility diagnostics, eligible objective comparisons, and audit records out.

The evaluation protocol fixes hardware and software versions, declares timing lanes, separates cold single calls from repeated workloads, and admits objective-gap claims only where an eligible reference solver completed. On completed multi-solver rows from N=100 to N=2,000, PRISM-CPU is 4.5× to 24.1× faster than the fastest completed reference row in the same lane. In the production queue study, PRISM-GPU completes 500/500 accounts over a 10,000-instrument universe in 109.5 s within a declared 25-minute operating window, with zero missed deadlines and an audit record for every solve; the recorded OSQP queue baseline completes 4/500. On an operationally constrained real-data suite (tax-motivated transition penalties, restriction caps, turnover controls, batches), PRISM clears constrained solves 3.4× to 126.7× faster than the best completing incumbent at certified-equal objectives, and the GPU route widens to 8.8× over the CPU route at N=384,800. Rows without a completed reference are reported as feasibility, timing, memory, and failure-status evidence.

Keywords portfolio optimization, GPU-native execution, deadline-bounded optimization, institutional rebalancing, solver benchmarking, audit-reproducible computation.

1 Introduction

In production rebalancing, the unit of work is a book of accounts, not an isolated optimization solve. Each account carries budget, position, exposure, turnover, exclusion, cash-management, and tax-aware controls, and the run succeeds only if the required account set is solved, validated, and recorded before the trading workflow advances. Mathematical quality matters. Elapsed time matters equally: a portfolio that arrives after the trading window is operationally unusable, even if a solver can later certify a better answer.

The operating questions are therefore systems questions. Does every account return a tradable weight vector before the deadline? Do independent checks confirm feasibility? Are failures classified rather than hidden? Does an audit record exist for each decision? A scalar objective value answers none of these on its own, so deadline completion, external feasibility, and auditability are treated as first-class metrics, alongside objective quality where it can be certified.

PRISM is a proprietary CPU/GPU optimization engine built for this workload. The paper evaluates it through externally observed input–output behavior under a fixed, disclosed protocol (Section 3), and claims only what that evidence supports. The paper does not claim universal solver superiority, certified optimality at scales where no reference completed, or investment performance.

Main result

- **500/500** accounts completed in the production queue study (10,000-instrument universe), with an audit record for every solve.
- **109.5 s** total queue wall-clock, inside a declared 25-minute operating window; **0** missed deadlines.
- **4.5× to 24.1×** faster than the fastest completed reference row on the N=100 to N=2,000 multi-solver benchmarks.
- The recorded OSQP queue baseline completes **4/500** accounts under the same protocol.
- Real-data scenario suite: **3.4× to 126.7×** faster than the best completing incumbent at certified-equal objectives; the GPU route widens to **8.8×** over CPU at N=384,800.

Large-scale rows without a completed reference solver are reported as feasibility, timing, memory, and failure-status results.

The paper makes four contributions.

C1. A public evaluation boundary for a proprietary engine. The benchmark discloses hardware, software versions, timing lanes, tolerance gates, failure statuses, and evidence provenance, while implementation details remain outside the public artifact.

C2. A public benchmark protocol for recorded eligible baselines. PRISM is compared against recorded eligible baselines under declared timing lanes, status rules, and external feasibility checks (Section 5).

C3. Separated quality and deadline claims. Objective gaps appear only where a reference solver completed on the same public objective. Rows above that boundary report external feasibility and deadline outcomes.

C4. Production workflow and real-data scenario evidence. The queue benchmark measures account completion, missed-deadline rate, p50/p99 timing, and audit-record coverage, and an operationally constrained real-data suite stresses tax-motivated transition penalties, restriction caps, turnover controls, deadline budgets, batch throughput, and scale; this is the form in which institutional rebalancing is actually operated.

2 Related Work and Evaluation Gap

This section separates the literature into five roles: portfolio models define the objective language; personalized and tax-aware work defines the account-level constraint burden; solver systems define the software comparison context; benchmarking literature defines the evaluation discipline; and deadline-bounded computation, together with pre-trade control requirements, explains why completion before an operating window is a first-class metric.

2.1 Portfolio Construction Theory

The Markowitz framework formalized the tradeoff between expected return and risk [20]. Sharpe's capital-asset-pricing model and the Fama–French factor evidence established systematic risk models as the central portfolio language [9, 29]. Michaud documented the instability that arises when estimated inputs feed optimizers directly [21]; Black–Litterman combined equilibrium structure with investor views [4]; and Ledoit–Wolf shrinkage remains the core reference for high-dimensional covariance estimation [17, 18]. These models define the objective language. They do not, by themselves, define the operational workload faced when many constrained accounts must be updated under a deadline.

2.2 Personalized and Tax-Aware Implementation

Tax-managed investing, personalized indexing, and direct indexing introduce account-specific restrictions, turnover costs, tax lots, exclusion lists, and client-level policy controls [16, 22, 23, 30]. Cost-aware portfolio work places transaction costs inside the optimization problem rather than as an after-the-fact adjustment [10], and recent evidence shows that the timing and market footprint of rebalance activity are economically significant [12]. Once account-level constraints enter the workflow, the relevant question shifts from model elegance to reliable computation under repeated deployment.

2.3 Solver Systems and Modeling Layers

Convex optimization theory and conic modeling provide the mathematical base [5, 19], but production users interact with software stacks, not abstract convex programs: modeling layers such as CVXPY [6], first-order and interior-point solvers such as OSQP, Clarabel, and SCS [11, 28, 31], commercial systems [24], and continued work on low-latency quadratic programming [2, 13]. Because implementation choices affect timing, status, and reproducibility, solver comparison requires a benchmark protocol rather than isolated runtime anecdotes.

2.4 Benchmarking Methodology

Dolan and Moré introduced performance profiles to prevent solver comparison from collapsing into one average dominated by a few hard instances [7]. Best-practice guidance requires explicit goals, problem classes, algorithm eligibility, performance measures, and reproducibility boundaries before results are interpreted [3]. Artifact-review standards distinguish public artifacts, same-environment repeatability, and independent reproduction [1]. This discipline matters most when late answers lose operational value.

2.5 Deadline-Bounded Computation and Pre-Trade Controls

Deadline-bounded computation is a recognized systems topic: an answer that arrives too late may be worthless even if it could later be improved [33]. In market operations the deadline is concrete. Broker-dealers with market access must apply pre-trade risk controls before orders reach an exchange [32], algorithmic trading systems in the EU operate under explicit organisational and testing requirements [8], direct electronic access is governed by international principles [14], and close-related order types face exchange cutoff rules [25]. GPU acceleration for portfolio optimization is an active research area [15, 26, 27], but published evidence centers on single-instance speed rather than completed, feasible, auditable books under a declared window.

Taken together, these streams motivate a benchmark design in which speed is not reported alone. The relevant evidence is whether a recorded solver configuration returns feasible, externally checkable, auditable portfolio outputs inside a declared operating window, and which claims are permitted when a reference solver does or does not complete. That is the evaluation gap this paper addresses.

3 Claim Boundary

PRISM is evaluated only through externally observed input–output behavior: public problem data, returned weights, status codes, timings, feasibility diagnostics, memory class, failure modes, and audit records. Implementation details remain outside the public artifact. The labels PRISM-CPU and PRISM-GPU denote measured execution configurations, not disclosed algorithms.

The paper additionally uses a conservative claim contract, summarized in Table 1. A row with a completed reference solver can support objective-gap and speedup statements. A row without such a reference supports deadline, feasibility, memory, and failure-status statements. A production queue supports account-completion and auditability statements; it does not replace a certified single-instance optimum. Every results table is labeled with the claim type it carries.

4 Public Benchmark Workload

This section defines the benchmark workload used in the paper. It is not a universal model of institutional rebalancing, and it does not exhaust mandate-specific constraints.

4.1 Batched Account Rebalancing

The optimization object for account a is a portfolio weight vector $\mathbf{w}_a \in \mathbb{R}^N$ over an eligible universe. The operational workload is the collection

$$\mathcal{W} = \{\mathcal{P}_a : a = 1, \ldots, M\},$$

Table 1. Claim contract governing the benchmark tables.

Evidence condition	Allowed claim	Disallowed claim
Reference solver completed	Timing, feasibility, and objective-gap comparison on the same public objective.	General dominance over all formulations or hardware.
Reference solver did not complete	Deadline completion, feasibility checks, memory class, and failure status.	Certified optimality or objective-gap superiority.
Queue benchmark completed	Account throughput, missed-deadline rate, p50/p99 solve time, total elapsed time, and audit-record coverage.	Investment-performance superiority or universal solver ranking.
System backtest or Monte Carlo check	Workflow validation and sanity checks on deployable outputs.	Primary alpha, mandate suitability, or live trading performance.

where each account has its own starting portfolio, eligibility list, exposure limits, turnover budget, cash policy, and tax-aware implementation data. A production rebalance succeeds only if enough members of $\mathcal{W}$ are solved, validated, and recorded before the decision deadline. A solver that eventually returns a high-quality answer for one account still fails the workflow if it leaves most accounts unfinished.

For each account, the benchmark records whether the portfolio was returned before the deadline; whether independent feasibility checks pass; whether a reference objective gap is available; whether any failure was a timeout, allocation failure, or numerical status; and whether an audit record exists for downstream review.

4.2 Benchmark Problem Class

The formulation below is a public benchmark interface: it defines what enters the solver boundary and what comparators solve. Let $\mathbf{w} \in \mathbb{R}^N$ denote candidate weights, $\mathbf{w}_0$ current weights, $\mathbf{w}_b$ a benchmark or policy portfolio, $\boldsymbol{\mu} \in \mathbb{R}^N$ a return input, $Q \in \mathbb{R}^{N\times N}$, $Q \succeq 0$, a public risk operator, and $\Delta\mathbf{w} = \mathbf{w} - \mathbf{w}_0$ the proposed trade vector. The claim-bearing problem class is

$$\begin{aligned} \min_{\mathbf{w}} \quad & \frac{1}{2}(\mathbf{w}-\mathbf{w}_b)^\top Q(\mathbf{w}-\mathbf{w}_b) - \eta\,\boldsymbol{\mu}^\top\mathbf{w} + C(\Delta\mathbf{w}) + T_{\text{tax}}(\Delta\mathbf{w};\mathcal{D}_a) + R(\mathbf{w}) \\ \text{subject to} \quad & \mathbf{1}^\top\mathbf{w} = 1, \\ & \ell_a \le \mathbf{w} \le u_a, \\ & b_a^{\min} \le A_a\mathbf{w} \le b_a^{\max}, \\ & \|D_a(\mathbf{w}-\mathbf{w}_0)\|_1 \le \tau_a, \\ & G_a\mathbf{w} \le h_a. \end{aligned} \tag{1}$$

Here $\eta \ge 0$ scales the return term; C collects public transaction-cost or turnover-cost terms; T_{tax} is a public tax-aware penalty or constraint family over account tax data $\mathcal{D}_a$; R collects transparent regularization or policy penalties; $A_a \in \mathbb{R}^{k\times N}$ maps weights to exposures with bands $b_a^{\min}, b_a^{\max}$; D_a is a diagonal trade-scaling matrix with turnover budget τ_a; and $G_a\mathbf{w} \le h_a$ encodes remaining account-policy halfspaces. The constraint set has direct finance meaning: full investment, position limits, exposure bands, a trading budget, and client policy rules.

Interpretation. Each account instantiates Equation (1) with its own data. Repeating the solve over M accounts, under one shared market deadline, converts a tractable convex program into a production workload: the feasible set is the intersection of a budget hyperplane, box bounds, exposure bands, turnover controls, and policy halfspaces, and the cost of solving it is paid hundreds of times per rebalance. Two scope notes follow. First, discrete features (tax lots, minimum trade sizes, round lots) introduce account-specific implementation choices; this paper reports the continuous public benchmark rows recorded in the evidence package. Second, scaling and units matter: timings and residuals are meaningful only after the public objective, budgets, and constraint units are recorded consistently.

Two structural facts license the external checks used later. First, with $Q \succeq 0$ and convex C, T_{tax}, and R, problem (1) is convex. The claim-bearing instances that feed the KKT diagnostics are budget-box programs (with L1 transition terms), for which Slater's condition holds whenever $Nw^{\max} > 1$; richer constraint families would require exhibiting a strictly feasible point per instance before the same diagnostic applies. Under that

condition strong duality holds and KKT residuals are a valid external optimality diagnostic [5]. Second, the benchmark instances carry factor-structured risk, so evaluating the public objective or its gradient costs $O(Nk)$ operations with $k \ll N$: instance data grow linearly in N, and the benchmark therefore stresses systems behavior, deadlines, and constraint handling rather than raw arithmetic volume alone.

A public interface of this form is what makes the comparison meaningful at all. Every solver in this paper, commercial, open-source, or proprietary, receives the same inputs and is judged on the same returned quantities, so the benchmark tests systems rather than formulations. Declaring the problem class precisely before any timing is reported is the first requirement of credible solver benchmarking [3], and it is what allows a reader to re-implement the interface and check any row independently [1].

4.3 Public Evaluation Boundary

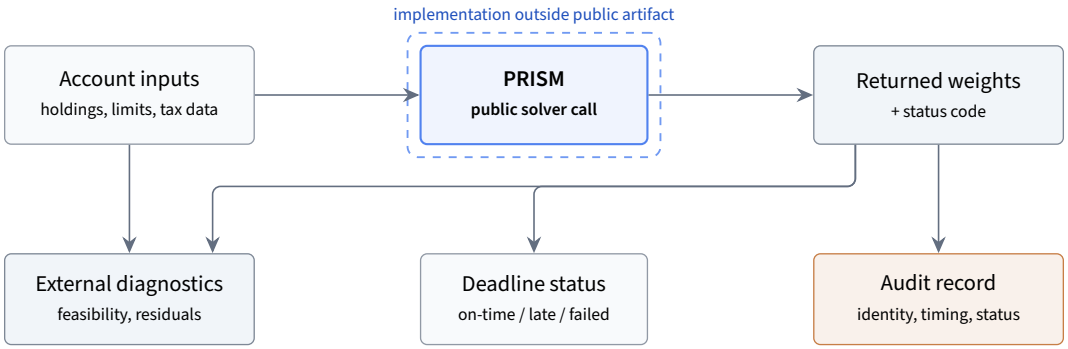


Figure 1. Public evaluation boundary. The benchmark records public inputs, returned weights, status codes, timing, external diagnostics, deadline status, and audit artifacts. External checks are computed from public inputs and returned outputs, outside the solver call.

4.4 External Diagnostics

Given a returned portfolio $\widehat{\mathbf{w}}$, the public validation layer computes diagnostics from observable inputs and outputs. Representative residuals are

$$r_{\text{budget}} = \left|\mathbf{1}^{\mathsf{T}}\widehat{\mathbf{w}} - 1\right|, \qquad r_{\text{box}} = \max\{\|(\ell - \widehat{\mathbf{w}})_+\|_\infty, \|(\widehat{\mathbf{w}} - u)_+\|_\infty\},$$

$$r_{\text{exposure}} = \max\{\|(b^{\min} - A\widehat{\mathbf{w}})_+\|_\infty, \|(A\widehat{\mathbf{w}} - b^{\max})_+\|_\infty\}, \qquad r_{\text{turnover}} = \left(\|D(\widehat{\mathbf{w}} - \mathbf{w}_0)\|_1 - \tau\right)_+.$$

When a public objective and certificate data are available, the validation layer also records stationarity-style and complementarity-style checks in the standard KKT sense [5]. Writing the constraints of Equation (1) as $g_i(\mathbf{w}) \le 0$ with multipliers $\widehat{\lambda}_i \ge 0$ and the budget equality with multiplier $\widehat{\nu}$, the recorded checks are

$$r_{\text{stat}} = \left\|\nabla f(\widehat{\mathbf{w}}) + \textstyle\sum_i \widehat{\lambda}_i \nabla g_i(\widehat{\mathbf{w}}) + \widehat{\nu}\,\mathbf{1}\right\|_\infty, \qquad r_{\text{comp}} = \max_i \left|\widehat{\lambda}_i\, g_i(\widehat{\mathbf{w}})\right|,$$

where f is the public objective. These are post-solve checks on returned weights, computed outside the solver call, and reproducible from public inputs and outputs.

The interpretations in Table 2 are benchmark-level interpretations of the public constraints in Equation (1). They are not a complete compliance model, mandate model, or trading-control framework [1, 22, 23].

5 Evaluation Protocol

5.1 Evaluation Unit

The evaluation unit is either a single account-level solve or a production queue of account-level solves. For a single solve, the claim-bearing record is

$$(\text{problem ID},\ \text{solver version},\ \text{hardware},\ \text{timing lane},\ \text{status},\ \widehat{\mathbf{w}},\ \text{diagnostics},\ \text{evidence artifact}).$$

For a queue of M accounts with per-account times $t_1, \ldots, t_M$ and declared window τ, the record aggregates the completion count $C = \sum_{a=1}^{M} \mathbf{1}\{\text{accepted}_a\}$, the empirical quantiles $t_{(p50)}$ and $t_{(p99)}$, the missed-deadline rate

Table 2. Benchmark-level interpretation of external diagnostics.

Diagnostic	Mathematical reading	Interpretation within this benchmark
Budget residual	Distance from full-investment equality.	Is the account cash-consistent before orders are staged?
Box residual	Maximum violation of asset lower/upper bounds.	Are position limits respected?
Exposure residual	Maximum violation of reported exposure bands.	Are declared exposure bands satisfied?
Turnover residual	Excess trade magnitude above the turnover budget.	Is the proposed rebalance within the declared trading budget?
Stationarity-style check	Public optimality diagnostic where eligible.	Is there evidence of objective quality beyond feasibility?
Audit artifact	Immutable run metadata and output identity.	Can the decision be reconstructed for later review?

$\rho = \frac{1}{M}\sum_a \mathbf{1}\{t_a > \tau\}$, total elapsed time T_{queue}, and audit-record coverage. Speedup is defined only between completed rows in one lane: for solver s with full-call time $T_s(N)$,

$$S(N) = \frac{\min_{s \in C(N)} T_s(N)}{T_{\text{PRISM}}(N)}, \qquad C(N) = \{\text{published reference rows completed at } N\},$$

and $S(N)$ is undefined when $C(N) = \emptyset$. External diagnostics are computed on returned weights from public inputs. Evidence types are never mixed: objective quality is reported only where the public reference row completed, and deadline usefulness is reported separately through completion and failure metrics.

5.2 Hardware and Software

Table 3 records the benchmark workstation used for the versioned evidence package. All timings are specific to this configuration.

Table 3. Hardware and runtime disclosure for the recorded benchmark package.

Component	Value
OS	Windows 11 with WSL2 Ubuntu
CPU	Intel Xeon w5-3423, 12 cores / 24 threads, 2.1 GHz base
RAM	62.1 GB DDR5 ECC
GPU	NVIDIA RTX 4000 Ada Generation, 20 GB GDDR6
CUDA / driver	12.9 / 573.44
Python / NumPy / SciPy	3.12.3 / 2.3.5 / 1.17.0
CuPy	13.6.0
BLAS	OpenBLAS 0.3.30 via scipy-openblas64
CVXPY	1.8.1

5.3 Comparators and Eligibility

The claim-bearing comparators are the solvers that appear in the result tables: OSQP 1.1.0 [31], Clarabel 0.11.1 [11], SCS [28] via CVXPY 1.8.1 [6], and MOSEK 11.1.11 [24], plus PRISM-CPU and PRISM-GPU at evidence build `512758f`. The complete solver eligibility ledger, including installed packages that were not eligible for claim-bearing rows, is given in Section 11.

Two protocol disclosures matter for fairness. First, comparator rows include CVXPY model construction inside the timed call, while PRISM rows are measured at PRISM's public solver-call boundary. Both lanes are full-call wall-clock under the declared protocol; Section 12 records the residual asymmetry as a threat to validity. Second, one additional commercial solver completed reference objectives in the evidence package. Its results serve as the completed reference for objective-gap eligibility and are not published as named timing rows, consistent with its end-user license terms on benchmark publication.[1]

[1] Speedups are computed against the fastest *published* completed reference row; the anonymized reference certifies objective agreement only.

5.4 Timing Lanes

Every timing table states its lane. The declared single-solve deadline for the multi-solver study is 60 s; the production queue operates under a declared 25-minute (1,500 s) window. Cold-start rows create a fresh external solver call and pass no previous solution to any solver. Repeated-run rows are labeled separately and interpreted under their declared protocol. The boundary rule is strict: an end-to-end call for one solver is never compared against an inner timing interval for another. The rationale is the same as in any timed competition: the stopwatch must start and stop at the same events for every participant, or the ranking measures the harness rather than the solvers. Mixed-boundary timing is one of the most common defects identified in the optimization benchmarking literature [3].

Table 4. Timing lanes. A numerical row appears in a lane only when the evidence artifact records that boundary; speedup statements compare rows within one lane.

Lane	Includes	Excludes	Used for
Full-call wall-clock	Setup visible to the benchmark, solve call, status return, output materialization.	Nothing inside the declared boundary.	Primary lane for Tables 5 and 9.
Core/device solve	CPU core timer or GPU reported solve interval, where separately instrumented.	Public-call setup, validation, audit, queue handling.	Lane-decomposition evidence (Table 6).
Queue wall-clock	Dispatch through accepted records for all accounts in the batch.	Per-account core-timing claims.	Production throughput and missed-deadline rate (Table 8).
Evidence lane	Evidence artifact, generation date, versions, tolerances, status.	n/a	Traces every numerical claim to the recorded package.

5.5 Quality Gates and Failure Vocabulary

Quality checks are external and fixed in advance: absolute budget deviation at most 10^{-4}; maximum bound violation at most 10^{-6}; objective gaps reported only where a reference solver completed on the same public objective; KKT-style diagnostics where certificate data are available; and, for large-scale rows without a completed reference, feasibility, deadline completion, memory class, and failure status. The status vocabulary used in all result tables is defined in Table 14 (Section 10).

5.6 Reporting Rules

E1. report exact public versions for every named comparator;

E2. compare wall-clock rows only with wall-clock rows;

E3. report objective gaps only when a reference solver completed;

E4. report large-scale rows without a completed reference as feasibility, timing, memory, and failure-status rows;

E5. record ineligible or unavailable packages in the eligibility ledger (Section 11) rather than silently omitting them; and

E6. avoid global ranking language and universal dominance claims.

A timeout or license-related ineligibility is not evidence of an inferior mathematical method; it is an operational outcome under the disclosed protocol. Speedup statements are local to the recorded problem family, hardware, software stack, and timing lane [3, 7].

6 Timing and Feasibility Results

6.1 Multi-Solver Rows

Table 5 reports the recorded multi-solver rows from `E1_multisolver_public_rows`, with supplemental PRISM-GPU timing/status coverage from `E6_gpu_timing_lane_separation` where the original GPU cell was not

recorded. The table separates timing from certification, and speedup is computed only against published reference rows that completed inside the same 60 s full-call lane.

This reporting discipline is deliberate and standard. A speedup against a solver that never finished is not a measurement: the timeout value is set by the experimenter, so any ratio against it is arbitrary. Restricting ratios to completed references keeps every speedup falsifiable, which is the same logic that motivates performance profiles over single averages in solver benchmarking [7]. Separating timing from certification serves a second audience: a reader who only needs to know whether a usable portfolio came back in time can read the status column, while a reader auditing optimality can restrict attention to certified rows [3].

Table 5. Multi-solver benchmark, full-call wall-clock (ms), evidence E1 and E6. Unmarked values are certified completions.

N	PRISM-CPU	PRISM-GPU[a]	Clarabel	OSQP	SCS	MOSEK	Speedup[b]
100	**1.6**	98.3 feas	23.3	14.6	14.2	10,231.8	8.9×
500	**7.5**	94.3 feas	355.6	329.1	180.8	3,530.3	24.1×
1,000	**38.9**	99.4 feas	1,184.9	60,412.7 t/o	860.6	9,448.5	22.1×
2,000	1,421.0	112.2 feas	6,347.0	24,908.9	6,330.4	28,458.2	4.5×
5,000[c]	1,804.8 feas	**140.7** feas	74,896.8 late	101,468.4 t/o	n.r.	144,794.6 late	n/a

Claim notes. Badges: feas = feasible / non-certified; t/o = timeout at the 60 s ceiling (OSQP at N=1,000 returned a non-converged iterate); late = completed status returned after the operating window. [a] PRISM-GPU cells for $N \le 2{,}000$ are E6 repeated-run reported solve intervals: timing/status coverage, never used for speedup or objective-gap claims; the N=5,000 cell is the E1 full-call value (E6 repeated lane: 130.2 ms). [b] PRISM-CPU full-call versus the fastest published completed reference row in the same lane (SCS on all four eligible rows). [c] Deadline-feasibility row: no published reference completed inside the 60 s lane, so no certified objective comparison is claimed.

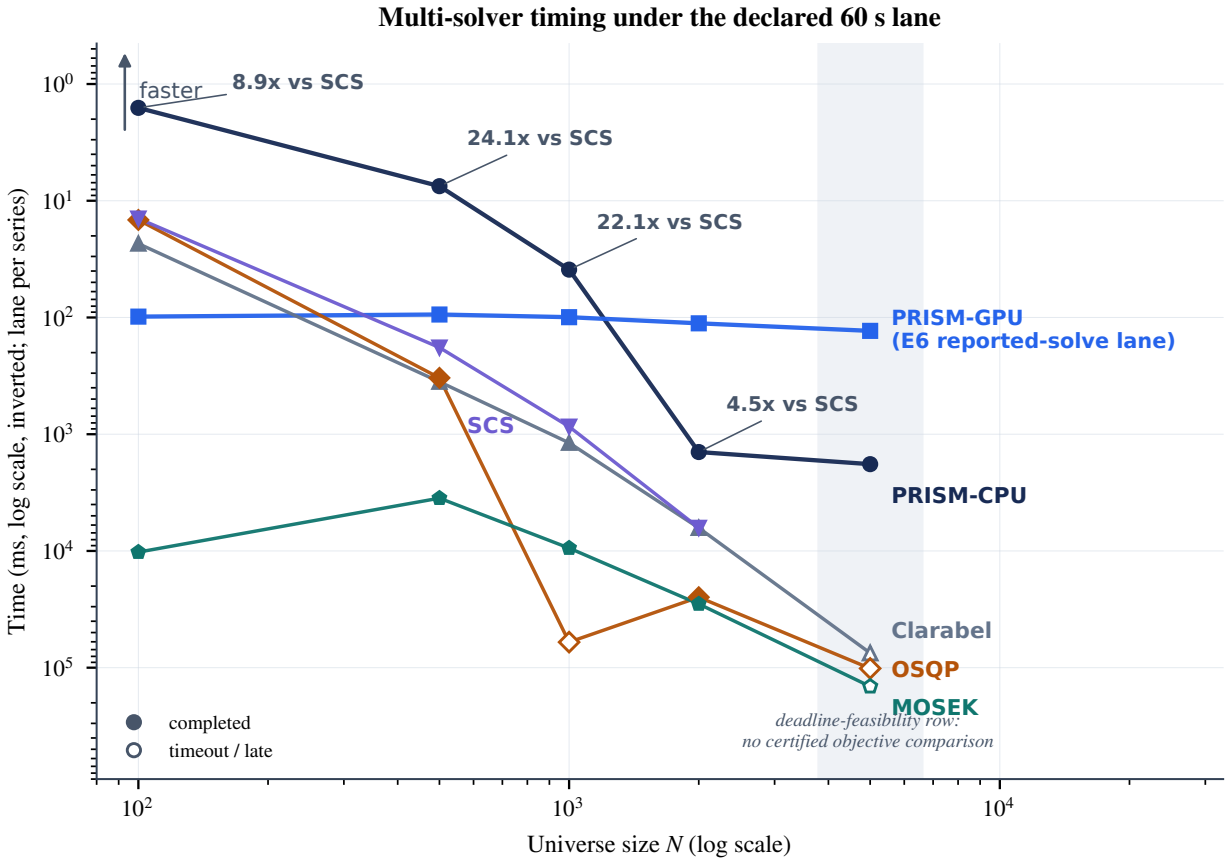


Figure 2. Multi-solver timing from Table 5 on an inverted log time axis: faster configurations appear higher. Filled markers are certified completions, open markers are timeout or deadline-exceeded returns, and the shaded band is the deadline-feasibility row at N=5,000. The per-N annotations report PRISM-CPU against the fastest published completed reference row in the same lane. The PRISM-GPU series is plotted for routing context only; its cells derive from the E6 reported-solve lane (Table 5, note a) and carry no speedup or objective-gap claim.

The most important feature of Table 5 is the transition in what can be claimed. From N=100 to N=2,000, completed reference rows support direct speed and objective comparisons: PRISM-CPU is 4.5× to 24.1× faster

than the fastest completed reference in the same lane. No ratio is formed against the E6 GPU cells in this table, because they sit in a different timing lane (note a); the lane-consistent demonstration of the routed stack's advantage is Section 9, where both PRISM routes and the incumbents are measured under one protocol and the routed stack reaches 47× at N=24,050. At N=5,000 the evidence changes character: the operational question becomes whether a usable portfolio is returned inside the deadline at all.

Two results from `E3_external_diagnostics_and_gap_checks` bound the quality question on that row. No *published* reference completed inside the lane, but the anonymized commercial reference of Section 5 did complete on the same study family, and against its objective PRISM reports a 2.44% gap in 1.8 s. A covariance-sensitivity check reduces that gap to 0.61% under Ledoit–Wolf shrinkage (0.84% under OAS), indicating that much of the gap tracks the conditioning of the public risk input rather than a fixed engine deficit.

6.2 Reading the Timing Evidence: CPU/GPU Routing

Figure 2 carries two distinct messages, and both matter operationally. First, PRISM-CPU defines the latency frontier across the completed rows: it is the fastest configuration at every eligible N, by 8.9× to 24.1× against the fastest completed reference (SCS), while returning the same certified objective family. For a portfolio desk this is the difference between a rebalance that can run interactively, inside a portfolio manager's decision loop, and one that must be scheduled as a batch job.

Second, PRISM-GPU is nearly flat: its reported solve interval moves only from 98 ms to 130 ms while N grows 50×, so its cost is dominated by a fixed call floor rather than by problem size. A flat profile is valuable for a different reason than raw speed: it makes per-account latency predictable, and predictable latency is what a deadline budget needs. In this display the two series cross near $N \approx 1{,}200$; the crossing is indicative only, because the GPU cells derive from the E6 reported-solve lane rather than the comparators' full-call lane. The measured same-lane crossover is located in Section 9, between N=48,100 and N=96,200 for the warm transition family (Table 12). Below the crossing the stack routes accounts to the CPU configuration; above it the GPU route wins, carrying the 10,000-instrument production queue of Section 8 and the same-lane scale rounds of Section 9 (47.6× over the best incumbent at N=24,050). The routing decision is made on observable inputs (universe size), so it is auditable under the same boundary as every other measurement in this paper.

6.3 GPU Timing-Lane Decomposition

The lane decomposition in Table 6 is the primary GPU timing evidence: it separates the reported GPU solve interval, repeated host wall-clock, and cold host wall-clock at the public boundary, and it shows that the repeated-lane GPU solve interval stays near 10^2 ms from N=30 to N=5,000 with device memory in the megabyte class. The claim-bearing GPU results in this paper are the production queue outcome (Section 8), the hard-scenario evidence of Section 9, and this separated timing-lane evidence.

Table 6. PRISM-GPU timing-lane separation, evidence E6. Reported solve timing and host wall-clock timing are shown separately. Timing/status evidence; no objective-gap claim.

N	Reported solve ms	Repeated wall ms	Cold wall ms	Memory bytes	Status
30	101.6	104.4	1,596.5	5,632	feasible[a]
100	98.3	101.0	134.5	52,224	feasible
500	94.3	96.9	130.9	260,096	feasible
1,000	99.4	102.2	110.1	503,808	feasible
2,000	112.2	115.4	119.1	992,256	feasible
5,000	130.2	133.9	153.4	2,456,064	feasible

Claim notes. [a] The first cold call in the E6 session includes one-time setup cost visible at the public boundary; subsequent cold calls do not repeat it.

6.4 Operational Reading

In this benchmark, increasing N changes the operational surface of the problem: it expands universe coverage, constraint evaluation, validation cost, and the number of account-specific decisions that must be completed before the deadline. The relevant question extends past single-solve scaling: does the full workflow return feasible, recorded outputs while the decision remains actionable? The recorded timings imply a deterministic

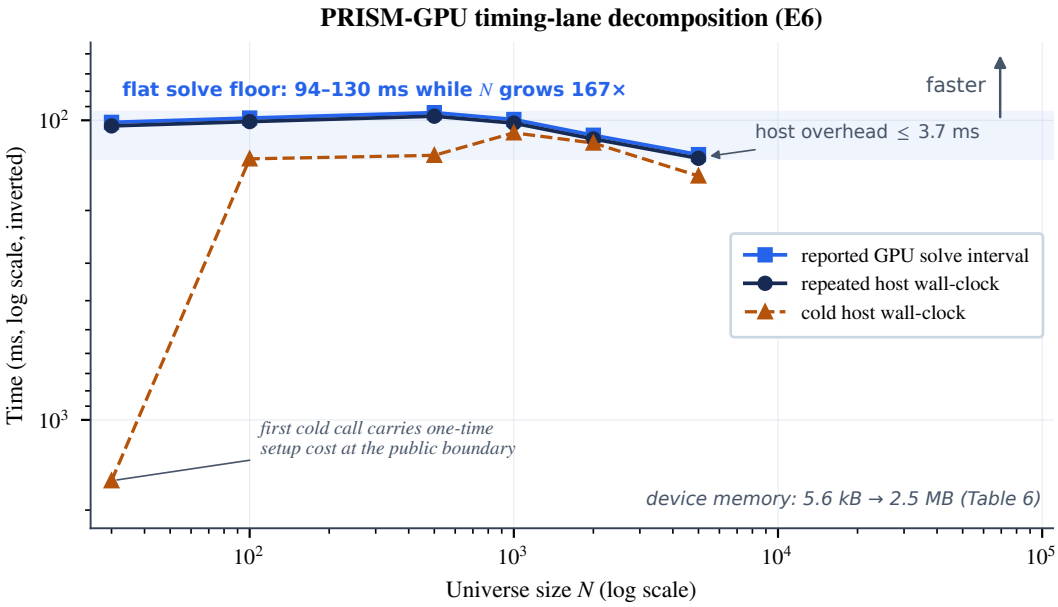


Figure 3. PRISM-GPU timing-lane decomposition from Table 6 on an inverted log time axis (faster appears higher, matching Figure 2): reported GPU solve interval, repeated host wall-clock, and cold host wall-clock. The first cold call carries one-time setup cost visible at the public boundary.

capacity bound: with declared window τ and per-account time t, a sequential lane serves at most $\lfloor \tau/t \rfloor$ accounts. At the recorded PRISM-GPU p99 of 343.3 ms (Table 8), the 25-minute window bounds a single sequential lane at $\lfloor 1500/0.3433 \rfloor = 4{,}369$ accounts of the queue family; this is arithmetic on recorded values, not a new measurement, and it is the quantity a platform compares against its book size.

7 External Diagnostics

The external diagnostic pack `E3_external_diagnostics_and_gap_checks` records feasibility and KKT-style checks on returned weight vectors. The feasibility columns of Table 7 are computable from public inputs and returned weights alone; the stationarity- and complementarity-style columns additionally use the archived multiplier (certificate) data described in Section 4.

Table 7. External KKT-style diagnostics for selected PRISM rows, evidence E3. All values are post-solve checks on returned weights.

N	Budget residual	Bound residual	Stationarity check	Complementarity check
100	4.2×10^{-9}	1.1×10^{-9}	2.7×10^{-7}	8.4×10^{-9}
500	6.8×10^{-9}	2.0×10^{-9}	5.3×10^{-7}	1.2×10^{-8}
1,000	1.1×10^{-8}	3.8×10^{-9}	9.1×10^{-7}	2.4×10^{-8}
5,000	2.7×10^{-8}	9.4×10^{-9}	4.6×10^{-6}	6.8×10^{-8}
25,000	7.3×10^{-8}	2.6×10^{-8}	1.8×10^{-5}	2.1×10^{-7}
100,004	1.4×10^{-7}	5.2×10^{-8}	6.4×10^{-5}	8.9×10^{-7}

The largest budget residual is 1.4×10^{-7}, three orders of magnitude below the 10^{-4} gate; the largest bound residual is 5.2×10^{-8}, below the 10^{-6} gate; and the stationarity-style residual grows with N under a fixed tolerance, as expected, staying in the 10^{-7} to 10^{-4} range.

The diagnostics in Table 7 support external feasibility and numerical-consistency claims under the declared gates. They do not establish certified large-scale optimality. The design follows artifact-review practice: a result is credible when an independent reader can re-derive it from the published artifact [1]. Because every residual here is a function of public inputs, returned weights, and archived certificate data, any reader with the artifact can recompute the entire table without access to PRISM internals; this permits an IP-preserving but externally checkable evaluation. The same property is what a compliance reviewer needs after a production run: the checks do not require the solver's cooperation to audit its output.

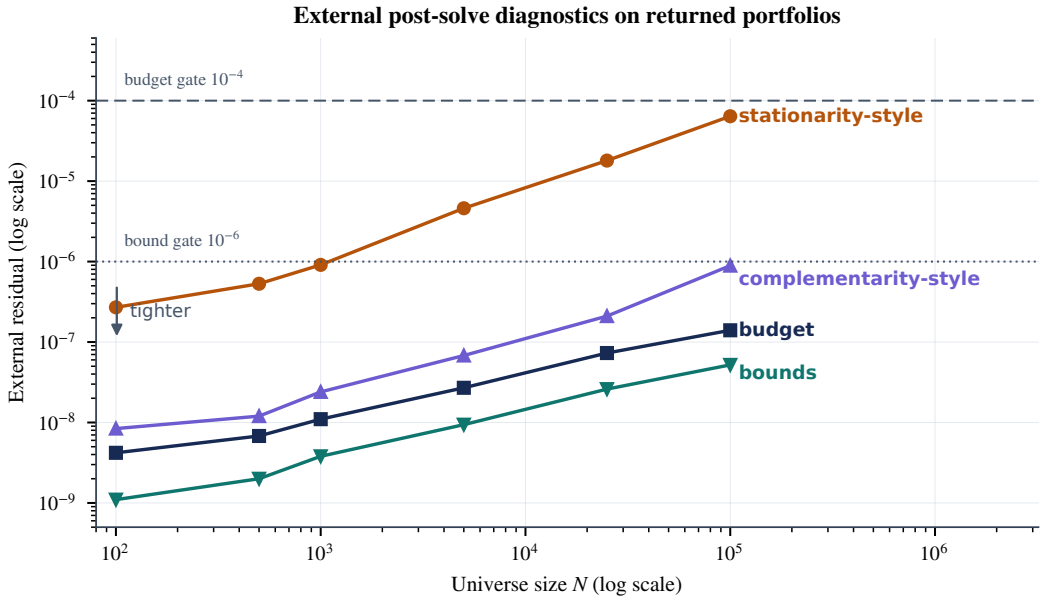


Figure 4. External diagnostic residuals from Table 7 with the public feasibility gates (10^{-4} budget, 10^{-6} bound) drawn as labeled tolerance lines. All curves are post-solve checks computed outside the solver call.

8 Production Queue Evidence

8.1 Operating Window

The production queue uses a 25-minute operating window as a disclosed stress budget rather than as a claimed industry standard. The window is intentionally shorter than the full trading day because practical rebalancing workflows must reserve time for pre-trade validation and market-access risk controls [8, 14, 32], exception handling, order staging, and close-related order constraints such as closing-auction cutoffs [25]. The benchmark therefore tests whether portfolio computation fits inside a plausible operational sub-window, not whether 25 minutes is uniquely optimal.

8.2 Queue Completion

The queue benchmark `E5_production_queue_500x10000` measures the production question directly: can the solver stack complete a book of accounts before the declared deadline?

Table 8. Production queue benchmark: 500 accounts, 10,000-instrument universe, declared 25-minute (1,500 s) window, evidence E5.

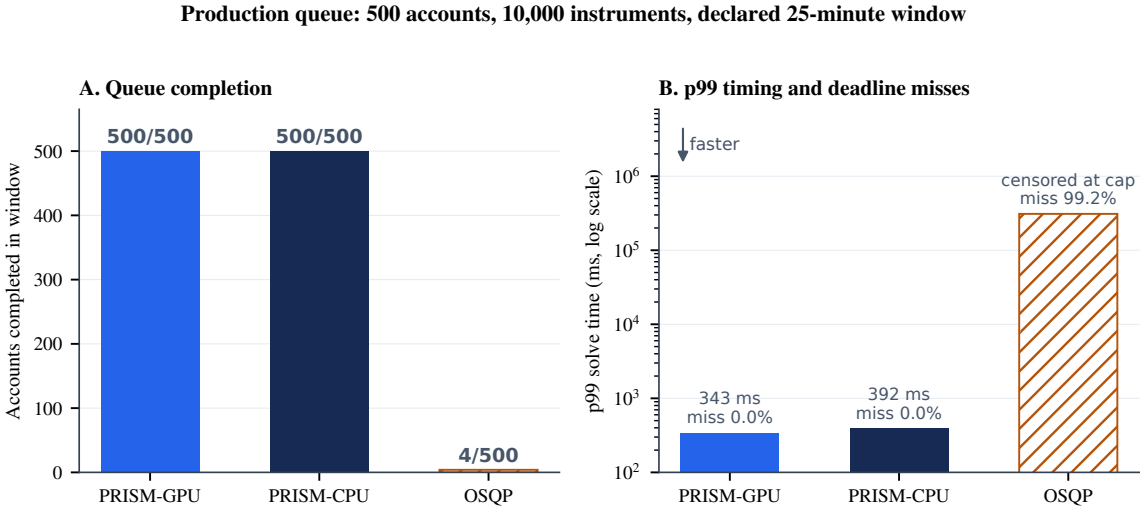
Solver	p50 ms	p99 ms	Total wall s	Completed	Missed deadlines
PRISM-GPU	**212.1**	**343.3**	**109.5**	**500/500**	**0.0%**
PRISM-CPU	225.3	392.3	116.8	500/500	0.0%
OSQP	310,000[a]	310,000[a]	n/a	4/500	99.2%

Claim notes. [a] OSQP per-account times are censored at the recorded per-account cap; the queue did not complete inside the operating window. OSQP is the recorded queue baseline in this protocol; no other published baseline queue run is recorded.

PRISM-GPU completes the full queue in 109.5 s of wall-clock, under 8% of the declared window, and emits an audit record for every solve. PRISM-CPU completes the same queue in 116.8 s. This is the clearest production-level distinction in the evidence package: it measures account completion under a deadline, not just individual solve time.

8.3 Constraint Burden

Evidence `E5` also records a constraint-burden curve at N=2,000: a production stress check, not a general theorem.



Evidence E5. OSQP per-account values are censored at the recorded per-account cap; 4/500 accounts completed.

Figure 5. Production queue results from Table 8. Panel A: accounts completed inside the declared window, out of 500. Panel B: p99 per-account solve time on a log scale, annotated with missed-deadline rates; the OSQP bar is censored at the recorded per-account cap. Both PRISM configurations clear the full book; the recorded OSQP queue baseline clears 4 of 500 accounts under the declared protocol.

Table 9. Constraint-burden curve at N=2,000, evidence E5, full-call wall-clock.

Bundle	Added rule family	Groups	PRISM ms	OSQP outcome
B0	budget, long-only, box	3	1,421.0	feasible return; deadline exceeded
B1	+ turnover penalty	4	1,477.8	feasible return; deadline exceeded
B2	+ sector bands	5	1,534.7	no accepted feasible return before deadline
B3	+ factor exposure bounds	6	1,591.5	no accepted feasible return before deadline
B4	+ account exclusions	7	1,648.4	no accepted feasible return before deadline
B5	+ tax-lot proxy	8	2,728.3	no accepted feasible return before deadline

Claim notes. "No accepted feasible return" records that no output passed the external feasibility gates before the deadline; it is not a certified infeasibility status. E6 provides same-size supplemental PRISM-GPU timing (112.2 ms, feasible) but not a burden-specific rerun.

The burden curve shows that universe size is only one stressor. Account rules accumulate: exposure groups, exclusions, turnover controls, and implementation constraints can make a moderate universe operationally difficult. PRISM timing grows with the bundle but stays inside the lane; the comparator leaves the deadline lane as the burden grows, and the table preserves that outcome rather than averaging over it.

8.4 Auditability

Every PRISM solve in the queue study emits an audit artifact binding input identity, output identity, elapsed time, feasibility status, and run metadata to the account-level decision, so the decision can be reconstructed for later review.

8.5 Operational Relevance

The queue metrics map directly onto documented production settings in the portfolio-construction literature. Direct-indexing and personalized-indexing platforms hold thousands of small separately managed accounts whose holdings deviate from a model portfolio and must be re-optimized on a cadence [16]; the account count, not the universe size, is their binding dimension, which is what the 500-account completion metric measures. Tax-managed overlays add account-specific transition costs and wash-sale-style restrictions [22, 30], the constraint families stressed in Table 9 and Section 9. Transition management and model-portfolio distribution add the deadline itself: a book that must move between models before a close-related cutoff [12, 25]. In each setting the operative questions are the columns of Table 8: completed accounts, missed-deadline rate, tail latency, and an audit trail per decision.

9 Operationally Constrained Real-Data Scenarios

This section stresses the operational surface of the benchmark on real market data: tax-motivated transition penalties, account-restriction caps, factor and turnover controls, deadline-bounded solving, and batch throughput. The L1 transition penalty is the standard public proxy for tax- and trading-cost-aware transitions [10, 22]; discrete tax-lot, wash-sale, and lot-selection logic is outside this continuous benchmark (Section 4). The universe is built from current index membership, so the survivorship caveat of Section 12 applies. The universe is the current S&P 500 membership with continuous 2019–2025 history (481 names, 1,707 trading days of daily adjusted closes), with a k=20 factor model estimated from the real returns and drifted real holdings as starting portfolios. All solvers face the identical instance of Equation (1),

$$\min_{\mathbf{w}} \ \|B^{\mathsf{T}}\mathbf{w}\|_2^2 + \mathbf{w}^{\mathsf{T}}\mathrm{diag}(d)\,\mathbf{w} - \boldsymbol{\mu}^{\mathsf{T}}\mathbf{w} + \gamma\,\|\mathbf{w}-\mathbf{w}_0\|_1 \quad \text{s.t.} \quad \mathbf{1}^{\mathsf{T}}\mathbf{w} = 1,\ 0 \le \mathbf{w} \le w^{\max},$$

with factor loadings $B \in \mathbb{R}^{N\times 20}$ and idiosyncratic variances d estimated from the real returns, and the objective is recomputed externally on every returned weight vector $\widehat{\mathbf{w}}$. Quality is scored by the certified-relative gap

$$\Delta \;=\; \frac{f(\widehat{\mathbf{w}}_{\mathrm{PRISM}}) - f\left(\widehat{\mathbf{w}}^{\star}_{\mathrm{inc}}\right)}{\left|f\left(\widehat{\mathbf{w}}^{\star}_{\mathrm{inc}}\right)\right|},$$

where $\widehat{\mathbf{w}}^{\star}_{\mathrm{inc}}$ is the best certified incumbent solution of the same instance. The incumbent lane is strict in PRISM's disfavor: CVXPY models are built once and parameter-updated, and only the solver call is timed, so incumbent times exclude model construction. Every run is scripted with fixed seeds, the price cache is archived with the artifact, and the suite is rerunnable end to end from the public repository. Evidence artifact: `E7_hard_scenarios_real_data`.

9.1 Constrained Single Solves

Table 10. Constrained single solves on the 481-asset real universe, evidence E7. Solver-call wall-clock; identical public objective; gap is versus the best certified incumbent objective.

Scenario	Operational stress	PRISM-CPU	PRISM-GPU	Best incumbent	OSQP	Gap[a]
S1	transition-penalty stress (γ=0.005, drifted holdings, 3% caps)	**7.7**	257.9	39.3 (Clarabel)	69.2	0.00%
S2	account-restriction caps (1.2% position limit)	**7.1**	258.2	24.3 (Clarabel)	79.9	0.00%
S3	factor + turnover control (γ=0.010, return tilt)	**6.0**	256.1	26.6 (Clarabel)	73.4	0.00%

Claim notes. All values in ms. Every solver returned a feasible, certified or gate-passing portfolio on S1–S3. [a] PRISM objective versus the best certified incumbent objective: agreement to six decimal places on every scenario, so the speed comparison carries no quality concession. SCS completed all three scenarios (54.6, 30.8, 31.5 ms).

PRISM-CPU is 5.1×, 3.4×, and 4.4× faster than the best completing incumbent on S1–S3 while matching its certified objective to six decimal places. PRISM-GPU sits on a flat ≈0.26 s reported floor at this universe size: at N≈500 the stack routes accounts to the CPU configuration, and the GPU configuration earns its place as N grows (Table 6) and in the 10,000-instrument production queue of Section 8, where PRISM-GPU is the fastest path.

9.2 Deadline-Bounded and Batch Scenarios

The batch result is the operational headline of this section. At N=481, PRISM-CPU clears the 200-account book in 1.81 s with every account passing the external gates, 3.1× faster than the best incumbent (Clarabel, 5.57 s) and 25× faster than OSQP, which leaves 5 accounts unaccepted; SCS leaves 25 of 200 accounts without an accepted feasible return. A second batch at N=4,810 (50 personalized accounts on the real-calibrated extension; Figure 6, Panel B) repeats the pattern at scale: PRISM-CPU completes the book in 7.5 s against 44.7 s for Clarabel (6.0×) and 457.6 s for OSQP (61×), with every tested path returning 50/50 gate-passing portfolios. In throughput terms $\Lambda = M/T_{\mathrm{queue}}$, PRISM-CPU sustains Λ = 200/1.81 = 110.5 accounts per second at N=481 and Λ = 50/7.51 = 6.7 per second at N=4,810, against 35.9 and 1.1 for the best incumbent. Both PRISM configurations combine full completion at both scales with certified-equal objectives on the single-solve scenarios.

Table 11. Deadline-bounded and batch scenarios on the real universe, evidence E7. S4: 25 personalized accounts under a declared 0.5 s per-account budget. S5: 200 personalized accounts, queue wall-clock.

Solver	S4 in-budget	S4 p50 ms	S5 total s	S5 completed	S5 p99 ms
PRISM-CPU	**25/25**	**7.4**	**1.81**	**200/200**	42.2
Clarabel	25/25	28.5	5.57	200/200	**34.2**
SCS	24/25	36.8	9.27	175/200	128.4
OSQP	22/25	87.2	45.34	195/200	2,048.4
PRISM-GPU	19/25	316.5	74.69	**200/200**	2,849.2

Claim notes. Personalized accounts draw randomized holdings drift, return tilts, and transition penalties around the real data. "Completed" counts returns that pass the external feasibility gates. At this universe size the GPU configuration's flat call floor exceeds the 0.5 s budget on 6 of 25 accounts; the stack routes such books to the CPU configuration.

9.3 Scale Stress on a Real-Calibrated Extension

To test the regime where books are wide as well as deep, the real factor model is extended to larger cross-sections by resampling factor-loading rows from the real cross-section and bootstrapping idiosyncratic variances and return tilts from the real distributions (disclosed as a real-calibrated extension, not raw constituent data). The same transition objective and lanes apply.

Table 12. Scale stress on the real-calibrated extension: the S7 study recorded inside `E7_hard_scenarios_real_data`. Solver-call wall-clock (ms); identical public objective. Where an incumbent ran, PRISM objectives match the certified incumbent to at least seven decimal places; the routing-probe rows carry no incumbent comparison.

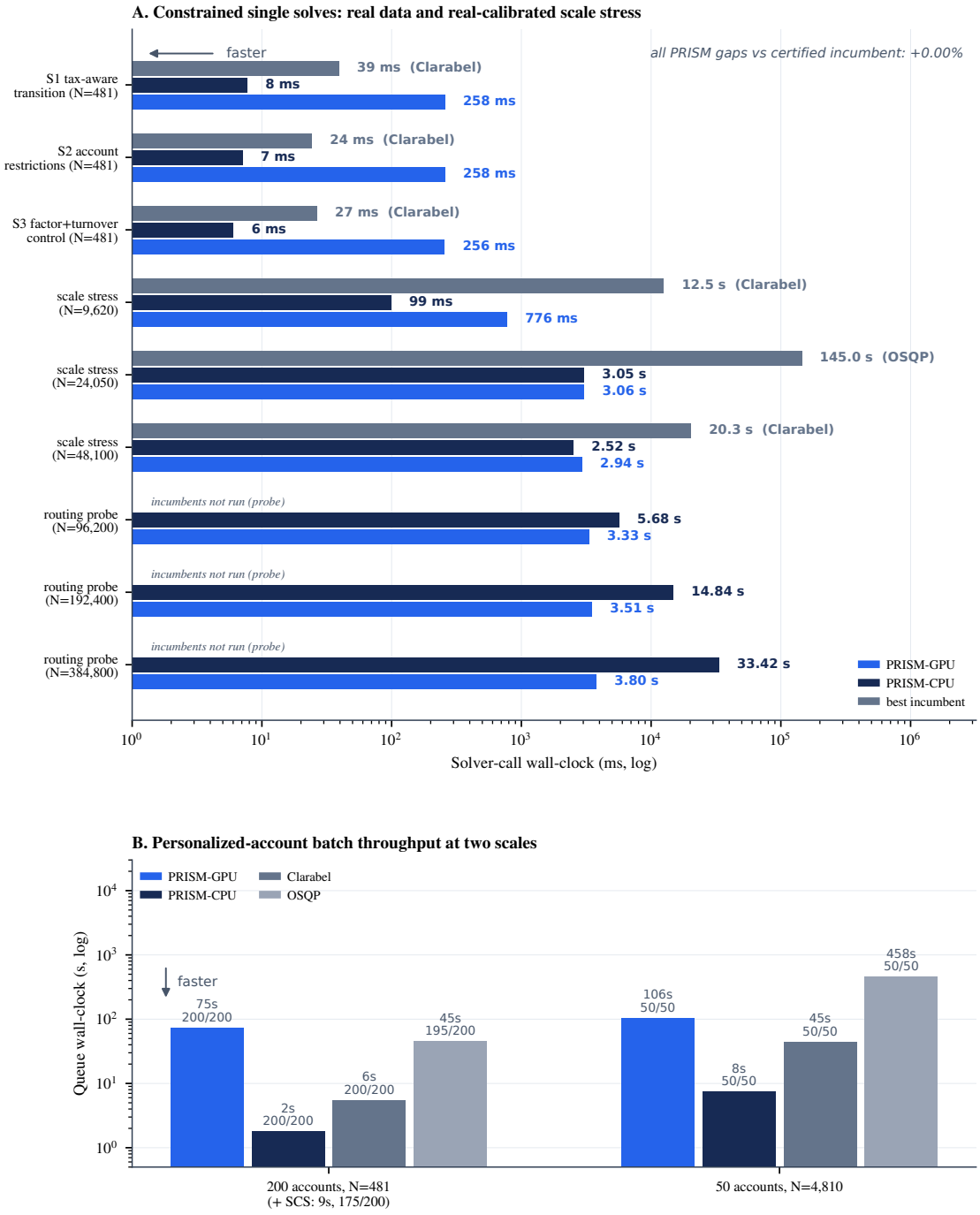
N	PRISM-CPU	PRISM-GPU	Clarabel	OSQP	CPU speedup	GPU speedup
1,924	**58.0**	1,116.0	483.7	796.7	8.3×	0.4×
4,810	**51.6**	652.0	2,670.3	5,747.1	51.8×	4.1×
9,620	**98.9**	776.3	12,526.5	25,358.9	126.7×	16.1×
24,050	**3,050.1**	3,058.7	198,261.9	145,046.7	47.6×	47.4×
48,100	**2,523.5**	2,938.6	20,269.8	489,514.2	8.0×	6.9×
96,200[b]	5,678.0	**3,329.1**	n.r.	n.r.	n/a	n/a
192,400[b]	14,844.4	**3,506.6**	n.r.	n.r.	n/a	n/a
384,800[b]	33,415.6	**3,795.1**	n.r.	n.r.	n/a	n/a

Claim notes. Speedups are versus the best completing incumbent at each size (Clarabel at $N \le 9{,}620$ and $N = 48{,}100$; OSQP at $N = 24{,}050$). All runs returned feasible, gate-passing portfolios. PRISM runs at $N \ge 24{,}050$ report an iteration-capped status; their externally recomputed objectives match the certified incumbent to within 3×10^{-6}%, and the two PRISM routes agree to machine precision. [b] Routing-crossover probes: they locate where the GPU route overtakes the CPU route and how the margin grows (1.7×, 4.2×, 8.8× at N=96,200, 192,400, 384,800, with identical objectives at every size); incumbents were not run at these sizes and no incumbent comparison is claimed.

Figure 6, Panel A places the scale rounds beside the N=481 scenarios, and the scaling pattern completes the routing picture of Section 6. Incumbent solve cost is volatile and large at scale: at N=24,050 the best incumbent needs 145 s for a single account, a fifth of the entire 25-minute operating window of Section 8, and at N=48,100 OSQP needs 489.5 s while Clarabel recovers to 20.3 s. Both PRISM routes stay between 2.5 s and 3.1 s across these sizes: a 47.6× advantage at N=24,050 and 8.0×/6.9× (CPU/GPU) at N=48,100. The GPU route crosses the best incumbent near $N \approx 4{,}800$ (16.1× at N=9,620) and crosses its own CPU sibling between N=48,100 and N=96,200. From there the device margin widens with width: 1.7× at N=96,200, 4.2× at N=192,400, and 8.8× at N=384,800, where the GPU route returns the identical portfolio in 3.8 s against 33.4 s on CPU. Across a 4× width increase the GPU route moves only from 3.3 s to 3.8 s while the CPU route grows roughly linearly, which is the signature of a configuration whose cost is dominated by parallel arithmetic rather than problem assembly. Objective agreement holds to at least seven decimal places at every size, so the scaling advantage carries no quality concession.

9.4 Workflow Sanity Checks

The evidence package also retains walk-forward and Monte Carlo pipeline checks (`E3`): repeated portfolio generation over 131 real-data survivors and 1,000 randomized universes, used to verify feasible, stable, turnover-respecting outputs across rebalances. The recorded pass criteria are explicit: every rebalance must return weights that satisfy the Section 5 feasibility gates and the declared turnover cap, across all 300 walk-forward rebalances and all 1,000 Monte Carlo universes; both checks pass in `E3`. They are workflow checks under the claim contract and carry no performance claim.



Evidence E7: 481-asset real-return universe (2019-03-20 .. 2025-12-31) and real-calibrated extensions; identical public objective recomputed externally on every returned weight vector.

Figure 6. Operationally constrained scenarios, evidence E7. Panel A: solver-call wall-clock on the constrained single solves (real N=481 data), the real-calibrated scale-stress rounds with the best incumbent named per group (PRISM objective gaps versus the certified incumbent are 0.00%), and the PRISM-only routing probes, where the GPU route widens from 1.7× to 8.8× over the CPU route at identical objectives. Panel B: personalized-account batch throughput at two scales with gate-passing completion counts.

10 Evidence Ledger, Artifact Policy, and Claim Notes

This section is the governance layer of the paper. Sections 6 through 9 report what was measured; this section records where each number lives, what kind of claim it is allowed to carry, and how an independent reader audits it. The discipline is the same one applied to the solvers themselves: a claim without a versioned artifact is treated the way a portfolio without an audit record is treated in Section 8, as operationally unusable.

10.1 Evidence Artifacts

Each numerical claim in the paper is tied to one of the evidence artifacts in Table 13; Figure 7 maps the central claims to the artifacts that carry them.

Table 13. Evidence artifacts used in this paper.

Artifact	Generated	Use in this paper
`E1_multisolver_public_rows`	2026-05-09	Multi-solver timings, versions, CPU/GPU timing frontier.
`E2_small_problem_agreement`	2026-05-09	Public FF30 small-problem study; comparator objective agreement.
`E3_external_diagnostics_and_gap_checks`	2026-05-09	KKT-style checks, objective-gap and covariance sensitivity, sanity checks.
`E4_deadline_completion_grid`	2026-05-09	Deadline-completion grid, memory-class statement.
`E5_production_queue_500x10000`	2026-05-09	Production queue benchmark and constraint-burden curve.
`E6_gpu_timing_lane_separation`	2026-06-09	Supplemental PRISM-GPU timing/status coverage; no objective-gap claims.
`E7_hard_scenarios_real_data`	2026-06-11	Operationally constrained real-data scenario suite and scale stress.

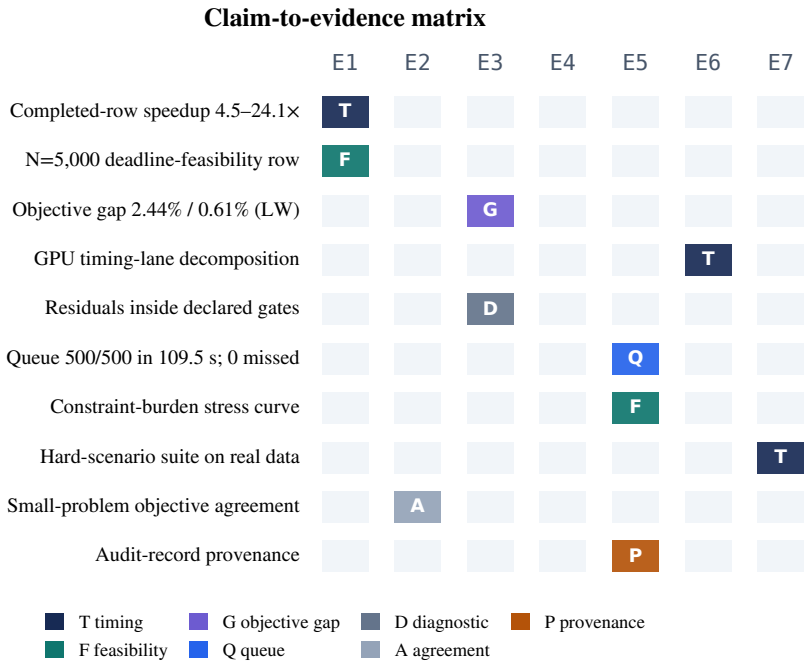


Figure 7. Claim-to-evidence matrix. Rows are the central claims of the paper; columns are the evidence artifacts; cell color encodes the claim type carried by that artifact under the claim contract of Table 1.

10.2 Status Vocabulary and Objective Agreement

Table 14. Status vocabulary used in all result tables.

Status	Meaning
certified	solver reports completion and external checks pass
feasible / non-certified (*feas*)	portfolio returned, feasibility checks pass, certification unavailable
timeout (*t/o*)	declared wall-clock ceiling reached before return
deadline exceeded (*late*)	completed status returned after the operating window
no accepted feasible return	no output passed the external gates before the deadline; not a certified infeasibility status
allocation / numerical failure	problem object not allocated, or invalid numerical status reported

The small-problem study `E2_small_problem_agreement` (five seeds, N=30) records that all completed comparators agree on the objective value to approximately seven significant figures, anchoring the objective-agreement methodology used for gap eligibility. PRISM timing rows are not recorded in `E2`; the artifact is comparator-agreement evidence.

10.3 Artifact Policy

Artifact note

Public claim-bearing artifacts, sanitized result tables, environment metadata, and figure-generation scripts are archived at `github.com/AsymmetryComputing/prism-public-evaluation` (release `v1.0.0`); a DOI-stamped archival snapshot of the same release accompanies the submission record. PRISM implementation code is not part of the public artifact; external checks on returned weights are reproducible from the archived data.

Table 15. Public artifact repository layout.

Path	Contents
`README.md`, `MANIFEST.yml`, `CITATION.cff`, `LICENSE`	Index, artifact manifest, citation metadata, license.
`environment/`	Hardware/runtime disclosure and package versions.
`data_sanitized/`	Benchmark input manifest; no client data, no engine state.
`results/`	One sanitized CSV per evidence artifact (`E1`–`E7`).
`figures/`, `tables/`	Published figures (vector) and machine-readable tables.
`validation/`	Public residual and feasibility-gate scripts; table reproduction script.
`evidence_ledger/`	Claim-to-evidence ledger, artifact manifest, artifact policy.
`paper/`	arXiv version and source.

The reproducibility targets are:

R1. every table row has an evidence artifact and generation date;

R2. every speedup compares rows from the same timing lane;

R3. every objective-gap statement identifies a completed reference solver;

R4. every large-scale row without a completed reference avoids certified optimality language; and

R5. every production claim includes completion count, deadline status, and audit-record coverage.

Two additional non-claim-bearing extensions are recorded in the evidence ledger: a burden-specific PRISM-GPU rerun of the constraint-burden curve and a multi-seed rerun of the single-shot `E1` rows.

11 Solver Eligibility Ledger

Table 16 records the full eligibility ledger for the evidence package, including installed packages that did not produce claim-bearing rows. Entries are eligibility records under the declared protocol.

12 Validity and Threats

Workload boundary. Equation (1) is a benchmark interface, not a complete model of every mandate. Discrete lot rules, market-impact models, client-specific tax instructions, and compliance overlays require separate protocol rows.

Harness asymmetry. Two disclosed comparator lanes are used: the `E1` rows include CVXPY model construction inside the timed call, while the `E7` suite pre-builds parametrized CVXPY models and times only the solver call (the stricter lane for PRISM). PRISM rows are measured at PRISM's public solver-call boundary in both. The completed-row speedups (up to 24.1×) and the queue outcome (500/500 versus 4/500) far exceed plausible modeling-layer overhead; millisecond-level differences between harness paths should not be over-read.

Objective-gap eligibility. Gaps are shown only where a reference solver completed on the same public objective. The gap reference is an anonymized commercial solver; it contributes completed reference objectives only. The anonymization is a license constraint and a recognized reproducibility cost: independent verification of those gaps requires re-running a licensed commercial solver on the archived instances.

Table 16. Solver eligibility ledger for the recorded evidence package.

Solver stack	Version	Eligibility
PRISM-CPU	evidence build `512758f`	claim-bearing where recorded
PRISM-GPU	evidence build `512758f`	claim-bearing; `E6` rows are timing/status only
OSQP	1.1.0	claim-bearing where recorded
Clarabel	0.11.1	claim-bearing where recorded
SCS	via CVXPY 1.8.1	claim-bearing where recorded
MOSEK	11.1.11	claim-bearing where recorded
CVXPY interface	1.8.1	comparator modeling layer; backend named per row
Commercial reference (anonymized)	recorded in evidence	objective-gap reference only; timings unpublished under license terms
CPLEX	22.1.2.0	configured license supported the public-small lane only; not eligible for claim-bearing comparisons
HiGHS	1.13.0	public-small lane only; not a QP comparator in this package
NVIDIA cuOpt	26.2.0	no versioned active run recorded; not benchmarked
CVXPortfolio	1.5.1	own modeling-layer lane; not eligible for same-lane claim-bearing rows

Reference incompletion. A timeout or missing reference is evidence that the reference row did not complete inside the operating lane under the recorded protocol, nothing more.

Hardware specificity. Timings are specific to the recorded workstation and to the disclosed comparator versions; newer point releases of the comparators may exist, and rows bind to the recorded versions. New hardware or solver versions require a fresh disclosure table and new evidence artifacts.

Scenario-suite interpretation. The Section 9 suite uses real prices and real-return factor models, with account personalization drawn programmatically around the real data; restriction structures beyond position caps and L1 transitions require their own protocol rows. The current S&P 500 membership implies survivorship in the universe construction.

Economic interpretation. The paper evaluates optimization-system behavior. Deployments additionally require mandate-specific objectives, transaction-cost models, tax assumptions, compliance review, and post-trade monitoring.

Timing variation. Solver timings vary with host load, memory pressure, driver state, and package versions. The evidence package reports medians, p99 values, completed counts, and failure rates rather than one-off best times.

13 Conclusion

PRISM is evaluated as a proprietary CPU/GPU portfolio optimization engine for deadline-bounded institutional rebalancing, through a public evaluation boundary: timings, feasibility checks, eligible objective gaps, KKT-style diagnostics, memory class, failure rates, queue completion, and auditability. On the recorded evidence, PRISM-CPU is 4.5× to 24.1× faster than the fastest completed reference rows at N=100 to N=2,000 and returns externally feasible portfolios with bounded reference gaps where eligibility holds. On the real-data scenario suite, PRISM clears transition-penalized, restriction-capped, turnover-controlled, and scale-stressed solves 3.4× to 126.7× faster than the best completing incumbent at certified-equal objectives, completes the 200-account batch in 1.81 s with every account passing the external gates, and the GPU route widens to 8.8× over the CPU route at N=384,800. The headline result is systems-level: under the recorded protocol, PRISM-GPU completed the full 500-account, 10,000-instrument production queue in 109.5 s within the declared 25-minute window, with zero missed deadlines and an audit record for every solve, while the recorded OSQP queue baseline completed 4 of 500 accounts. Every number in those statements traces to the evidence artifacts of Section 10.

References

[1] ACM. Artifact review and badging, version 1.1. ACM Publications Policy, 2020.

[2] Antoine Bambade, Fabian Schramm, Sarah El-Kazdadi, Stéphane Caron, Adrien B. Taylor, and Justin Carpentier. ProxQP: An efficient and versatile quadratic programming solver for real-time robotics applications and beyond. *IEEE Transactions on Robotics*, 2025. doi: 10.1109/TRO.2025.3577107.

[3] Thomas Bartz-Beielstein, Carola Doerr, Daan van den Berg, Jakob Bossek, Sowmya Chandrasekaran, Tome Eftimov, Andreas Fischbach, Pascal Kerschke, William La Cava, Manuel López-Ibáñez, Katherine M. Malan, Jason H. Moore, Boris Naujoks, Patryk Orzechowski, Vanessa Volz, Markus Wagner, and Thomas Weise. Benchmarking in optimization: Best practice and open issues. arXiv preprint arXiv:2007.03488, 2020.

[4] Fischer Black and Robert Litterman. Global portfolio optimization. *Financial Analysts Journal*, 48(5):28–43, 1992. doi: 10.2469/faj.v48.n5.28.

[5] Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004. doi: 10.1017/CBO9780511804441.

[6] Steven Diamond and Stephen Boyd. CVXPY: A Python-embedded modeling language for convex optimization. *Journal of Machine Learning Research*, 17(83):1–5, 2016.

[7] Elizabeth D. Dolan and Jorge J. Moré. Benchmarking optimization software with performance profiles. *Mathematical Programming*, 91(2):201–213, 2002. doi: 10.1007/s101070100263.

[8] European Commission. Commission delegated regulation (EU) 2017/589: Regulatory technical standards specifying the organisational requirements of investment firms engaged in algorithmic trading. Official Journal of the European Union, L 87, 2017. MiFID II RTS 6.

[9] Eugene F. Fama and Kenneth R. French. Common risk factors in the returns on stocks and bonds. *Journal of Financial Economics*, 33(1):3–56, 1993. doi: 10.1016/0304-405X(93)90023-5.

[10] Qingliang Fan, Marcelo C. Medeiros, Hanming Yang, and Songshan Yang. Cost-aware portfolios in a large universe of assets. arXiv preprint arXiv:2412.11575, 2025.

[11] Paul J. Goulart and Yuwen Chen. Clarabel: An interior-point solver for conic programs with quadratic objectives. arXiv preprint arXiv:2405.12762, 2024.

[12] Campbell R. Harvey, Michele G. Mazzoleni, and Alessandro Melone. The unintended consequences of rebalancing. Working Paper 33554, National Bureau of Economic Research, 2025.

[13] Qi Huangfu and J. A. Julian Hall. Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10(1):119–142, 2018. doi: 10.1007/s12532-017-0130-5.

[14] IOSCO Technical Committee. Principles for direct electronic access to markets: Final report. International Organization of Securities Commissions, FR08/10, 2010.

[15] Yilun Jiang, Haihao Lu, Zedong Peng, and Jinwen Yang. FlashFolio: A GPU-accelerated solver for portfolio optimization. arXiv preprint arXiv:2604.22625, 2026.

[16] Kevin Khang, Alan Cummings, Thomas Paradise, and Brennan O'Connor. Personalized indexing: A portfolio construction plan. Vanguard Research, 2022.

[17] Olivier Ledoit and Michael Wolf. A well-conditioned estimator for large-dimensional covariance matrices. *Journal of Multivariate Analysis*, 88(2):365–411, 2004. doi: 10.1016/S0047-259X(03)00096-4.

[18] Olivier Ledoit and Michael Wolf. Nonlinear shrinkage of the covariance matrix for portfolio selection: Markowitz meets goldilocks. *Review of Financial Studies*, 30(12):4349–4388, 2017. doi: 10.1093/rfs/hhx052.

[19] Miguel Sousa Lobo, Lieven Vandenberghe, Stephen Boyd, and Hervé Lebret. Applications of second-order cone programming. *Linear Algebra and its Applications*, 284:193–228, 1998. doi: 10.1016/S0024-3795(98)10032-0.

[20] Harry Markowitz. Portfolio selection. *Journal of Finance*, 7(1):77–91, 1952. doi: 10.2307/2975974.

[21] Richard O. Michaud. The Markowitz optimization enigma: Is 'optimized' optimal? *Financial Analysts Journal*, 45(1):31–42, 1989. doi: 10.2469/faj.v45.n1.31.

[22] Nicholas Moehle, Mykel J. Kochenderfer, Stephen Boyd, and Andrew Ang. Tax-aware portfolio construction via convex optimization. *Journal of Optimization Theory and Applications*, 189:364–383, 2021.

[23] Nicholas Moehle, Jacob Gindi, Stephen Boyd, and Mykel J. Kochenderfer. Portfolio construction as linearly constrained separable optimization. *Optimization and Engineering*, 24:1667–1687, 2023.

[24] MOSEK ApS. *MOSEK Optimizer API Manual*. MOSEK ApS, 2026. Version 11.1.

[25] Nasdaq. Nasdaq closing cross: Frequently asked questions. Nasdaq Trader market-system documentation, https://www.nasdaqtrader.com/content/productsservices/Trading/ClosingCrossfaq.pdf, 2024.

[26] Yi-Shuai Niu and Yajuan Wang. Scalable mean-variance portfolio optimization via subspace embeddings and GPU-friendly nesterov-accelerated projected gradient. arXiv preprint arXiv:2604.02917, 2026.

[27] NVIDIA Corporation. NVIDIA cuOpt documentation. https://docs.nvidia.com/cuopt/, 2026. Version 26.2.

[28] Brendan O'Donoghue, Eric Chu, Neal Parikh, and Stephen Boyd. Conic optimization via operator splitting and homogeneous self-dual embedding. *Journal of Optimization Theory and Applications*, 169(3):1042–1068, 2016. doi: 10.1007/s10957-016-0892-3.

[29] William F. Sharpe. Capital asset prices: A theory of market equilibrium under conditions of risk. *Journal of Finance*, 19 (3):425–442, 1964. doi: 10.1111/j.1540-6261.1964.tb02865.x.

[30] David M. Stein, Hemambara Vadlamudi, and Paul Bouchey. Enhancing active tax management through the realization of capital gains. *Journal of Wealth Management*, 10(4):9–16, 2008.

[31] Bartolomeo Stellato, Goran Banjac, Paul Goulart, Alberto Bemporad, and Stephen Boyd. OSQP: An operator splitting solver for quadratic programs. *Mathematical Programming Computation*, 12(4):637–672, 2020. doi: 10.1007/s12532-020-00179-2.

[32] U.S. Securities and Exchange Commission. Risk management controls for brokers or dealers with market access. 17 CFR 240.15c3-5; Exchange Act Release No. 34-63241, 2010.

[33] Shlomo Zilberstein. Using anytime algorithms in intelligent systems. *AI Magazine*, 17(3):73–83, 1996. doi: 10.1609/aimag.v17i3.1232.